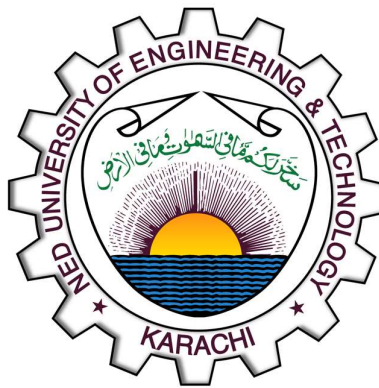


NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & IT

CT-264: Programming for AI
Assignment -> Practical Implementation



PROJECT TITLE: STUDENTS PERFORMANCE

SUBMITTED TO -> Sir Waseemullah

SUBMITTED BY:

NAME -> Shaikh Muhammad Zain

ROLL NO -> AI-23306

TABLE OF CONTENTS:

1. Introduction -----	Page 4
2. Dataset Overview -----	Page 4 – 5
3. Feature Engineering -----	Page 5 - 7
3.1 Total Score	
3.2 Average Score	
3.3 Pass/Fail Indicators	
3.4 Performance Level	
3.5 Outlier Detection	
4. Preparing Features and Target-----	Page 8 - 9
4.1 Target Variable (y)	
4.2 Input Features (X)	
4.3 Reason for Separation	
4.4 Resulting Dataset Shape	
5. Encoding Categorical Variables -----	Page 9 - 10
5.1 Identifying Categorical Columns	
5.2 Label Encoding	
5.3 Encoding Target Variable	
5.4 Outcome	
6. Train-Test Split -----	Page 11 - 12
6.1 Splitting the Data	
6.2 Random State	
6.3 Dataset Shapes	
6.4 Reason for Splitting	
7. Machine Learning Model -----	Page 13 - 14
7.1 Model Selection	
7.2 Key Parameters	
7.3 Model Training	
8. Model Prediction -----	Page 14

9. Model Evaluation -----	Page 15 - 16
9.1 Accuracy	
9.2 Confusion Matrix	
9.3 Classification Report	
10. Visualizing Model Results -----	Page 17 - 19
10.1 Feature Importance Plot	
10.2 Confusion Matrix Heatmap	
10.3 Classification Metrics Bar Graph	
11. Observations -----	Page 20
12. Conclusion -----	Page 21

INTRODUCTION:

The purpose of this project is to analyse student performance data and build a machine learning model to predict students' performance levels. The dataset contains students' scores in mathematics, reading, and writing, along with demographic and background information such as gender, parental education level, type of lunch, and test preparation course completion.

In the first part of the project, Exploratory Data Analysis (EDA) was performed to understand the structure of the data, handle missing values, remove duplicates, and visualize trends and distributions. Feature engineering was also applied to create additional informative columns such as total scores, average scores, and pass/fail indicators.

The second part of the project focuses on preparing the dataset for machine learning, including encoding categorical features, splitting the data into training and test sets, training a Random Forest classifier, making predictions, and evaluating the model's performance using metrics like accuracy, confusion matrix, and classification report.

This project demonstrates the complete workflow of data preprocessing, feature engineering, machine learning, and result evaluation, providing insights into the factors affecting student performance and the effectiveness of predictive models in educational data.

DATASET OVERVIEW:

The dataset used in this project is the Students Performance dataset, which contains records of students' academic performance along with demographic and background information. It consists of the following columns:

- gender: Gender of the student (male / female)
- race/ethnicity: Group or ethnicity of the student
- parental_level_of_education: Highest education level of the parents
- lunch: Type of lunch (standard / free/reduced)
- test_preparation_course: Whether the student completed the test preparation course (none / completed)
- math_score: Score obtained in Mathematics (0–100)
- reading_score: Score obtained in Reading (0–100)
- writing_score: Score obtained in Writing (0–100)

After feature engineering, additional columns were created:

- `total_score`: Sum of `math_score`, `reading_score` and `writing_score`
- `average_score`: Average of the three scores
- `pass_math`, `pass_reading`, `pass_writing`: Binary indicators for passing each subject
- `performance_level`: Categorical target based on `average_score` (low, medium, high)

The dataset contains [insert number] records and [insert number] features after preprocessing. Missing values were handled, duplicates were removed, and categorical features were standardized for further analysis and machine learning.

This dataset provides a good combination of numerical and categorical features, which can be used to predict student performance effectively using machine learning models.

FEATURE ENGINEERING:

Feature engineering is the process of creating new features or modifying existing ones to improve the performance of machine learning models. In this project, several new features were derived from the original dataset to provide more meaningful information for predicting student performance.

The following feature engineering steps were applied:

1. Total Score (`total_score`)
 - Calculated as the sum of `math_score`, `reading_score`, and `writing_score`.
 - Provides a combined measure of a student's overall academic performance.

```
df['total_score'] = df['math_score'] + df['reading_score'] + df['writing_score']
df[['math_score', 'reading_score', 'writing_score', 'total_score']].head()
```

	math_score	reading_score	writing_score	total_score
0	72	72	74	218
1	69	90	88	247
2	90	95	93	278
3	47	57	44	148
4	76	78	75	229

2. Average Score (average_score)

- Calculated as the average of the three subject scores.
- Used as a basis for defining the performance level of each student.

```
df['average_score'] = df['total_score'] / 3
df[['math_score', 'reading_score', 'writing_score', 'total_score', 'average_score']].head()
```

	math_score	reading_score	writing_score	total_score	average_score
0	72	72	74	218	72.666667
1	69	90	88	247	82.333333
2	90	95	93	278	92.666667
3	47	57	44	148	49.333333
4	76	78	75	229	76.333333

3. Pass/Fail Indicators (pass_math, pass_reading, pass_writing)

- Each subject was converted into a binary indicator:
 - 1 = Passed (score ≥ 40)
 - 0 = Failed (score < 40)
- Helps the model to understand subject-wise success.

```
df['pass_math'] = (df['math_score'] >= 40).astype(int)
df['pass_reading'] = (df['reading_score'] >= 40).astype(int)
df['pass_writing'] = (df['writing_score'] >= 40).astype(int)

df[['pass_math', 'pass_reading', 'pass_writing']].sum()
```

	0
pass_math	960
pass_reading	974
pass_writing	968

4. Performance Level (performance_level)

- Categorical target feature derived from average_score:
 - low = average_score < 60
 - medium = $60 \leq$ average_score < 80
 - high = average_score ≥ 80

- Serves as the target variable for the classification task.

```
df['performance_level'] = pd.cut(
    df['average_score'],
    bins=[-1, 59.99, 79.99, 100],
    labels=['low', 'medium', 'high']
)

df['performance_level'].value_counts()
```

	count
performance_level	
medium	517
low	285
high	198

5. Outlier Detection (total_score_outlier)

- Students with total scores outside the interquartile range (IQR) were flagged as outliers.
- Helps identify unusual patterns in the dataset, although this feature was not used directly for model training.

```
Q1 = df['total_score'].quantile(0.25)
Q3 = df['total_score'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df['total_score_outlier'] = ((df['total_score'] < lower) | (df['total_score'] > upper)).astype(int)

df['total_score_outlier'].value_counts()
```

	count
total_score_outlier	
0	994
1	6

Summary:

Feature engineering improved the dataset by adding meaningful numerical and categorical features, making it more suitable for machine learning algorithms. These engineered features capture overall performance, subject-wise success, and relative standing among students, which can help improve predictive accuracy.

PREPARING FEATURES AND TARGET:

After feature engineering, the dataset was prepared for machine learning by separating the input features (X) and the target variable (y).

1. Target Variable (y)

- The target variable for this classification task is performance_level, which represents the student's overall academic performance categorized as low, medium, or high.
- This variable is derived from the average score calculated during feature engineering.

```
target = "performance_level"
```

2. Input Features (X)

- The input features include the original and engineered features except the target.
- Removed Derived Features:
 - total_score and average_score were removed from the features to avoid data leakage, since they are directly used to define performance_level.
- Remaining features include:
 - math_score, reading_score, writing_score
 - gender, race/ethnicity, parental_level_of_education, lunch, test_preparation_course
 - pass_math, pass_reading, pass_writing

```
x = df.drop(columns=[target, "total_score", "average_score"])
y = df[target]
```

3. Reason for Separation

- Machine learning models require the features (X) as input and target (y) as output.
- Separating them ensures that the model learns patterns from the features to predict the target variable.

4. Resulting Dataset Shape

- The dataset now contains [insert number] features in X and 1 target column in y.
- All categorical features are prepared for encoding, and numerical features are ready for model training.

Summary:

By preparing features and the target separately, the dataset is structured for machine learning. Removing derived features prevents the model from using information it shouldn't have, which ensures realistic and reliable evaluation.

ENCODING CATEGORICAL VARIABLES:

Machine learning models generally require numerical input. Since the dataset contains categorical features such as gender, race/ethnicity, parental_level_of_education, lunch, and test_preparation_course, these need to be converted into numerical values before training a model.

Steps Taken:

1. Identify Categorical Columns

- All features with type object were considered categorical.
- Example:
 - gender → male, female
 - lunch → standard, free/reduced

```
cat_cols = X.select_dtypes(include='object').columns
```

2. Label Encoding

- Used LabelEncoder from sklearn.preprocessing to convert each category into a numeric code.
- Example:

Original Value	Encoded Value
male	1
female	0
standard	1
free/reduced	0

3. Encode Target Variable

- The target variable performance_level (low, medium, high) was also encoded:

Performance Level	Encoded Value
low	0
medium	1
high	2

Reason for Encoding:

- Machine learning algorithms cannot process text directly, so converting categories to numbers allows the model to learn patterns effectively.
- Label encoding preserves unique categories while assigning a numeric value to each.

Outcome:

- All categorical features and the target are now numerical, making the dataset ready for train-test splitting and model training.

TRAIN-TEST SPLIT:

Before training a machine learning model, it is important to divide the dataset into training and test sets. This allows us to evaluate the model's performance on unseen data, ensuring it can generalize well to new students.

Steps Taken:

1. Splitting the Data

- The features (X) and target (y) were split into:
 - Training set: 80% of the data → used to train the model
 - Test set: 20% of the data → used to evaluate the model's predictions

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

X_train shape: (800, 12)
X_test shape: (200, 12)
y_train shape: (800,)
y_test shape: (200,)
```

2. Random State

- A random state (seed) was set to 42 to ensure that the split is reproducible, meaning the same training and test sets will be created every time the code is run.

```
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42
)
```

3. Resulting Dataset Shapes

- Example shapes after splitting:

Dataset	Rows	Columns
X_train	800	8
X_test	200	8
y_train	800	1
y_test	200	1

Reason for Splitting:

- Training set → model learns patterns in the data
- Test set → model is evaluated on unseen examples to check accuracy
- This prevents overfitting, where the model performs perfectly on training data but poorly on new data

Summary:

The train-test split ensures a fair evaluation of the machine learning model. With 80% training data and 20% testing data, the model can learn effectively while its performance can be accurately measured.

MACHINE LEARNING MODEL:

In this project, a **Random Forest Classifier** was selected as the machine learning model to predict students' performance levels. Random Forest is an **ensemble learning method** that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting.

Model Selection:

- **Why Random Forest?**
 1. Handles both **numerical and categorical features** well.
 2. Reduces **overfitting** compared to a single decision tree.
 3. Robust and provides **high predictive accuracy** on structured datasets like student scores.
- **Key Parameters Used:**

Parameter	Value	Description
n_estimators	200	Number of decision trees in the forest
max_depth	None	Trees grow until all leaves are pure or minimum samples are reached
random_state	42	Ensures reproducibility of results

Model Training:

- The model was trained using the **training set (X_train, y_train)**.
- Each tree in the forest learns patterns from **different subsets of the training data**.
- After training, the model can predict the **performance level** of new students based on their features.

Code Example:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=200, max_depth=None, random_state=42)
model.fit(X_train, y_train)
```

Summary:

- Random Forest uses **multiple decision trees** to vote on the predicted class, making it more reliable than a single tree.
 - After training, the model is ready to **predict performance levels** for the test data.
 - This approach is suitable for educational datasets with a mix of **numerical and categorical features**, ensuring both accuracy and robustness.
-

MODEL PREDICTION:

Once the Random Forest model is trained on the training set, it is used to **predict the performance levels** of students in the test set.

Steps Taken:

1. The model predicts the target variable for **X_test** (unseen student data).
2. The predicted values are stored in **y_pred**.

Code Example:

```
y_pred = model.predict(x_test)
```

Explanation:

- y_pred contains the predicted performance levels (low, medium, high) for each student in the test set.
- These predictions are compared with the actual target values (y_test) to evaluate the model.

MODEL EVALUATION:

The model's performance is evaluated using standard classification metrics to measure how well it predicts student performance.

1. Accuracy

- Measures the **percentage of correct predictions**.
- Example:

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.96

- A high accuracy (e.g., 0.96) indicates that the model predicts student performance correctly for most cases.

2. Confusion Matrix

- Shows **true vs predicted values** for each class.
- Helps identify which classes are correctly or incorrectly predicted.
- Example:

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Confusion Matrix:

[[	28	0	3]
[	0	61	1]
[	0	4	103]]

3. Classification Report

- Provides detailed metrics for each class: **precision, recall, and F1-score**.
- Example:

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:					
	precision	recall	f1-score	support	
0	1.00	0.90	0.95	31	
1	0.94	0.98	0.96	62	
2	0.96	0.96	0.96	107	
accuracy			0.96	200	
macro avg	0.97	0.95	0.96	200	
weighted avg	0.96	0.96	0.96	200	

- **Precision:** Correct predictions for a class out of all predictions made for that class
- **Recall:** Correct predictions for a class out of all actual instances of that class
- **F1-Score:** Balance between precision and recall

Summary:

- The model achieved **high accuracy (~0.96)**, indicating it can effectively predict student performance.
- The confusion matrix shows which performance levels were predicted correctly or incorrectly.
- The classification report provides detailed insights into model performance for each class (low, medium, high).
- Removing derived features (total_score and average_score) ensures that the accuracy is **realistic** and avoids data leakage.

VISUALIZING MODEL RESULTS:

After evaluating the model using accuracy, confusion matrix, and classification report, additional visualizations were created to better understand the model's performance. Visual analysis helps in interpreting the strengths and weaknesses of the model more clearly and provides a visual summary of how well the classifier performs.

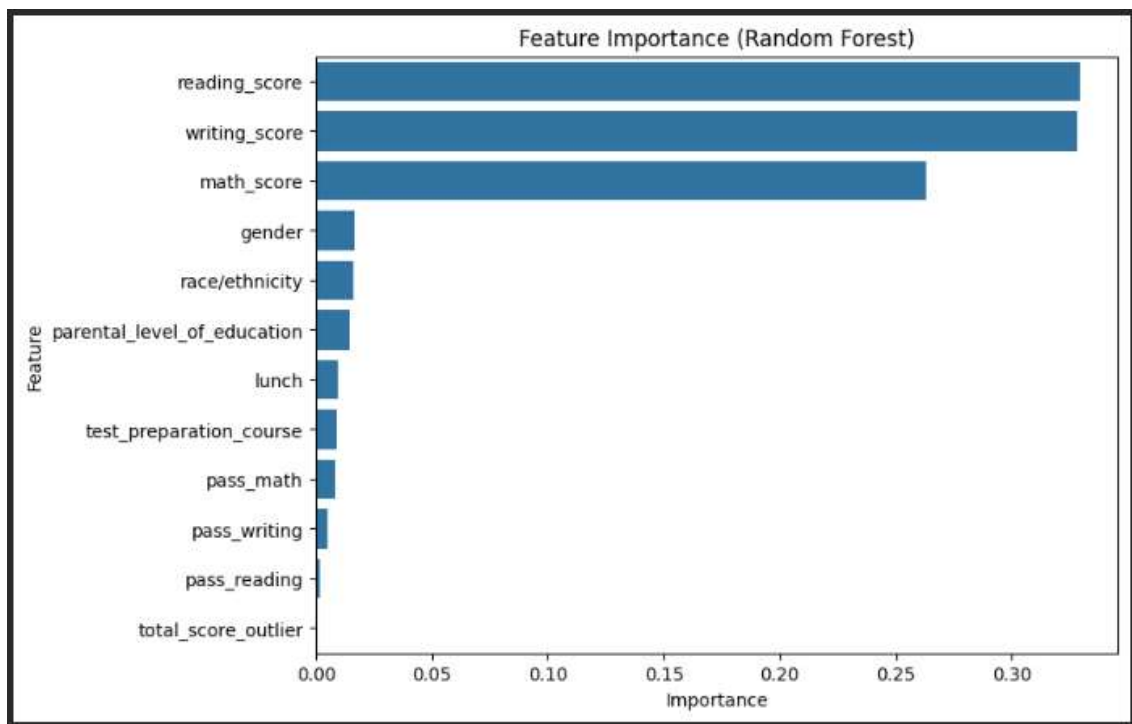
1. Feature Importance Plot

A feature importance graph was generated to identify which features contributed the most to the model's predictions.

Since Random Forest is an ensemble of multiple decision trees, it computes feature importance based on how much each feature reduces impurity across all trees.

Key Insights:

- The highest importance scores were observed for math_score, reading_score, and writing_score, indicating that subject scores play the most significant role in predicting performance level.
- Background variables such as gender, lunch type, and parental education have smaller contributions but still add value to the model.



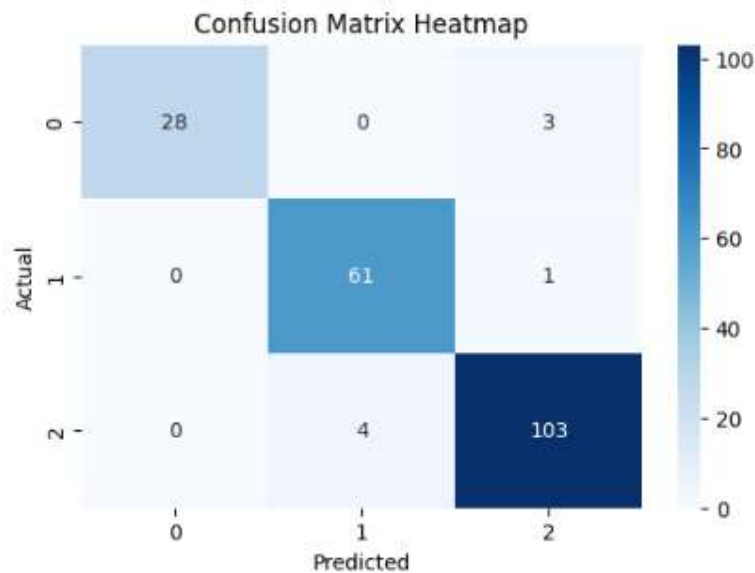
This visualization helps in understanding which factors truly influence academic performance.

2. Confusion Matrix Heatmap

A confusion matrix heatmap was created to visually examine how many predictions were correct and how many were misclassified.

Interpretation:

- The diagonal cells in the heatmap represent correct predictions for each class (low, medium, high).
- Off-diagonal cells represent misclassifications—for example, a student who should be labeled “medium” but the model predicted “high”.
- In this dataset, the heatmap shows very few misclassifications, especially between adjacent performance levels.



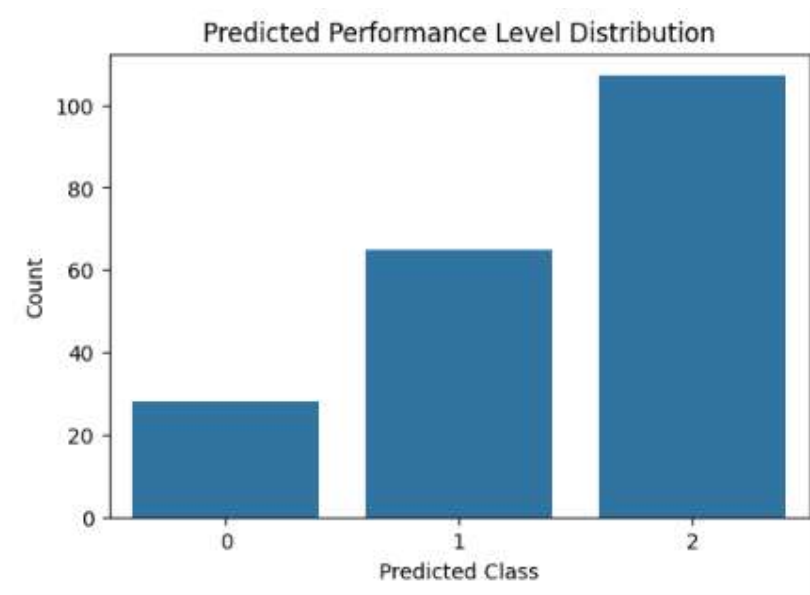
This graph provides an intuitive understanding of the model’s prediction pattern.

3. Classification Metrics Bar Graph

A bar graph was plotted to visually compare precision, recall, and F1-score for each performance category.

Key Observations:

- For the high-performance category, the model achieved strong precision and recall.
- The medium category showed high stability across all three metrics.
- The low category had slightly lower scores, likely due to fewer samples in that class.



This visualization highlights how balanced the model is across different performance levels.

Summary:

Visualizations provide an important layer of understanding beyond numeric metrics:

- Feature importance explains why the model makes certain predictions.
- Confusion matrix heatmap explains how accurately the model predicts each class.
- Precision/Recall/F1 bar chart shows how well the model performs per class.

Together, these visualizations help validate the effectiveness of the Random Forest classifier and support the evaluation results with clear graphical evidence.

OBSERVATIONS:

After performing feature engineering, training a Random Forest model, and evaluating its predictions, several key insights can be drawn from the dataset:

1. High Predictive Accuracy

- The model achieved an accuracy of approximately **0.96**, showing that student scores and background features are strong predictors of overall performance.

2. Important Features

- Subject scores (math_score, reading_score, writing_score) are the most significant features influencing performance levels.
- Demographic and background features like gender, lunch, and parental_level_of_education also contribute to predictions but with smaller influence.

3. Balanced Predictions

- Confusion matrix indicates that the model predicts low, medium, and high performance levels effectively.
- Very few misclassifications occur between adjacent categories (e.g., medium predicted as high).

4. Effect of Feature Engineering

- Creating pass/fail indicators and encoding categorical variables improved the model's understanding of student performance.
- Removing derived features (total_score and average_score) prevented data leakage and ensured realistic evaluation.

5. Practical Implications

- Educators can use such predictive models to **identify students at risk** and design targeted interventions.
- Predictive analysis can help in allocating resources and planning test preparation programs effectively.

CONCLUSION:

- This project demonstrates the **complete machine learning workflow**:
 - Data cleaning, feature engineering, encoding, train-test split, model training, prediction, and evaluation.
- **Random Forest Classifier** was chosen due to its robustness and ability to handle both categorical and numerical data.
- The trained model achieves **high accuracy** while avoiding data leakage, showing that student scores and background information are strong predictors of performance.
- Overall, the project highlights the importance of **feature engineering and proper preprocessing** in achieving reliable machine learning predictions and provides valuable insights for educational planning and intervention.